

Executable Evidence for Testing Differentiable Numerical Components: Strategy Increments and External Defect Cases

Ning Hu^{a,*}, Haitao Duan^b, Shuqun Li^c, Chuyang Hu^d

^a*School of Mechanical Engineering, Hangzhou Dianzi University, No. 1158, 2nd Street, Qiantang District, Hangzhou 310018, Zhejiang, China*

^b*School of Computer Science, Hangzhou Dianzi University, No. 1158, 2nd Street, Qiantang District, Hangzhou 310018, Zhejiang, China*

^c*School of Mathematics and Physics, Xi'an Jiaotong-Liverpool University, 111 Ren'ai Road, Suzhou Industrial Park, Suzhou 215123, Jiangsu, China*

^d*Department of Computer Science and Engineering, University of California, 1156 High Street, Santa Cruz, CA 95064, USA*

Abstract

Differentiable numerical components cross software and hardware interfaces in simulation and learning systems, but a successful call or autodiff trace does not show that their values, parameter derivatives, or execution policies are reliable. We develop an executable testing and qualification method for a batched component $y = P(x; \theta)$. Each evidence record fixes the operating domain, independent references, execution policies, provenance, tolerances, and scope before evaluation. Under the same synthetic ten-class catalogue, four cumulative strategies detected 3, 5, 8, and 10 classes; the complete record detected 200/200 injected faults and rejected 0/40 clean records. A PyTorch external-subject pilot increased detection from 25.00% to 100% over 240 injected trials. A matched value/property subset spanning NumPy, SciPy,

*Corresponding author. ORCID: Ning Hu 0000-0002-2718-0385; Haitao Duan 0009-0006-4611-8639; Shuqun Li 0009-0007-1787-685X; Chuyang Hu 0009-0007-9255-018X.

Email addresses: huning@hdu.edu.cn (Ning Hu), duanhaitao@hdu.edu.cn (Haitao Duan), Shuqun.Li24@student.xjtlu.edu.cn (Shuqun Li), chu207@ucsc.edu (Chuyang Hu)

and PyTorch increased detection from 28.57% to 100% over 252 injected trials and accepted 36/36 controls. Representation equivalence derived from SciPy #8906 detected 32/32 variants missed by a weak padded example. Unchanged runners reproduced PyTorch #80770 and SciPy #8906 and #15620 faults in official release wheels and accepted current fixes, completing three historical buggy/fixed pairs across two projects and three failure modes. The results support incremental executable evidence across controlled projects and real release-level defects, without implying broad field prevalence.

Keywords: software quality, numerical reliability, mutation testing, differentiable programming, test oracles, verification and validation

1. Introduction

Engineering software now couples learned modules to propagators, matrix-function actions, integration steps, and structured residual blocks. These components move between APIs, command-line tools, devices, dtypes, and continuous-integration jobs. Automatic differentiation follows the executed graph (Baydin et al., 2018), but does not remove discretization, truncation, or floating-point error. A component can satisfy a callable interface while violating numerical, unit, batch-isolation, or execution-policy requirements.

Such defects are often discovered after integration, when diagnosis, re-training, and repeated system tests are expensive. The missing interface is not another solver API. It is a portable statement of what was tested, on which operating domain, under which execution policy, and with what decision.

We address this need with an executable conformance specification. Its schema binds an operating domain and execution request to value, gradient, batch, repeatability, OOD, horizon, dtype/device, unit, provenance, calibration, and composition evidence. A stable record interface carries identity, version, provenance, execution, and decision fields; backend plugins supply the numerical controls required by a component class. Core, extended, and application profiles let a consumer request only the evidence needed at a particular integration boundary.

The work contributes (i) a versioned component-level evidence model and three cumulative qualification profiles, (ii) a reference implementation with canonical JSON, explicit requested-versus-observed execution fields, and

legacy-record migration, and (iii) paired strategy comparisons on both a synthetic fault catalogue and an upstream-defect-derived case. We ask three empirical questions: *RQ1*, which fault classes are added by numerical-property, execution-aware, and complete evidence strategies; *RQ2*, does the ordering transfer from record mutations to external numerical interfaces; and *RQ3*, can a representation-equivalence property expose a real historical defect pattern that a correct single example masks? The method is designed for research and engineering workflows that require portable, machine-checkable reliability evidence.

2. Related Work and Positioning

Differentiable programming has made numerical simulators and differential-equation solvers trainable components. Representative directions include automatic differentiation as a general computational technique (Baydin et al., 2018), differentiable physical simulation (Hu et al., 2020; Holl et al., 2020), neural ordinary differential equations (Chen et al., 2018), universal differential equations (Rackauckas et al., 2020), physics-informed neural networks and their software ecosystems (Raissi et al., 2019; Lu et al., 2021), and neural operators that learn maps between function spaces (Kovachki et al., 2023). These methods demonstrate the utility of differentiating through scientific computation, but they do not by themselves define a common acceptance test for the value, parameter derivative, calibration, and reuse behavior of an individual numerical component.

Existing ecosystems operate at complementary levels. DeepXDE and TorchPhysics organize differential-equation models, training objectives, geometry, and parameter-identification workflows (Lu et al., 2021; Tanyu et al., 2023); the SciML/DifferentialEquations ecosystem exposes common solver interfaces and work-precision benchmarking across algorithms and applications (Rackauckas and Nie, 2019). Our unit of qualification is one executed primitive. Before integration into a model or solver, the protocol checks independent value and derivative references, dtype-specific gates, and selected compositions, then writes the evidence and any operating restrictions to a machine-readable record.

Engineering verification and validation (V&V) evaluates a model or simulation against its requirements, intended use, referent data, uncertainty, and life-cycle evidence (National Aeronautics and Space Administration, 2024). The present workflow addresses a smaller unit: an executed differentiable

component. It checks the forward map, parameter-derivative path, batched and device behavior, and selected compositions under a declared protocol. A component-level **CONFORMANT** decision concerns the numerical software layer. Model form, uncertainty, and agreement with experiments remain end-to-end V&V questions.

The standards literature supplies adjacent concepts but no ready-made record for differentiable numerical kernels. ISO/IEC 25010 defines software-product quality characteristics, while ISO/IEC 25023 defines measures used to evaluate product quality (International Organization for Standardization, 2023, 2016). ISO/IEC/IEEE 29119-2 specifies organizational and project test processes, and IEEE 1012 addresses system, software, and hardware V&V (International Organization for Standardization, 2021; IEEE Standards Association, 2024). Table 2 maps our component-level fields to these sources. The mapping establishes traceable design provenance and identifies the component-level concerns implemented by the schema.

Numerical analysis supplies the diagnostics used by the software gates. Forward error, backward error, conditioning, and floating-point stability describe different failure mechanisms (Higham, 2002). Stable algorithms exist for matrix-exponential actions and Frechet derivatives (Al-Mohy and Higham, 2011, 2009), and derivative condition estimates quantify perturbation amplification (Higham, 2008; Higham and Relton, 2014). Discretized fractional calculus (Lubich, 1986), Mittag–Leffler evaluation (Garrappa, 2015; Gorenflo et al., 2014), and fractional Laplacian definitions (Lischke et al., 2020) provide corresponding references for fractional propagation. We use these established results to select independent evaluators, perturbation probes, and acceptance tolerances. Agreement between two autodiff traces is insufficient when both traces pass through the same inaccurate evaluator.

Our preceding DFSC study introduced an error-controlled differentiable Mittag–Leffler propagation component and its SciML interfaces (Hu et al., 2026). That work asks how a particular fractional propagator can be evaluated and differentiated reliably. The present paper asks what evidence must accompany a differentiable component before exchange or admission into a workflow. MLSL supplies one of four test backends, but no MLSL algorithm or numerical result is claimed here as a new contribution. Table 1 makes this boundary explicit. The reporting design follows reproducible-computing practice (Wilson et al., 2017); SciPy and other standard software provide independent controls (Virtanen et al., 2020).

Table 1: Boundary between the preceding component paper and the present conformance study.

Work	Primary object	Contribution used in the present paper
DFSC/MLSL (Hu et al., 2026)	Error-controlled Mittag-Leffler propagation and differentiable SciML interfaces	One implemented backend and its independent numerical controls
This study	Versioned evidence record, cumulative profiles, migration rules, and executable conformance decision	Component-exchange specification, standards traceability, coverage guards, and mutation validation

3. Executable Conformance Specification

In this paper, *conformance* is the agreement of a component record with the selected profile and its declared gates. A *specification* is the versioned record schema together with its validation rules; a *profile* is a cumulative set of required evidence; a *gate* is a predeclared Boolean criterion; and an *evidence plugin* produces backend-specific measurements. A *test action* executes one or more such measurements, whereas *qualification* denotes the resulting profile-level decision. These terms define the decision semantics implemented and evaluated in this work.

Each backend exposes a batched map

$$y = P(x; \theta), \quad (1)$$

with an explicit input and parameter domain, output shape, dtype policy, device policy, and reference evaluator. The protocol does not require every primitive to share the same mathematics. It requires the evaluation record to preserve the domain on which each claim is valid.

The executable specification stored with a submitted primitive is

$$\mathcal{C} = (\mathcal{D}_x, \mathcal{D}_\theta, R, \boldsymbol{\tau}, \mathcal{M}, \mathcal{S}, \mathcal{Q}, \mathcal{E}), \quad (2)$$

where $\mathcal{D}_x$ and $\mathcal{D}_\theta$ are the declared state and parameter domains, R is an independent reference evaluator, $\boldsymbol{\tau}$ contains gate tolerances, $\mathcal{M}$ records

Table 2: Relationship between the proposed component record and established standards. The final column states the narrower implementation in this work.

Source	Relevant concern	Component-level realization
ISO/IEC 25010	Functional suitability, reliability, performance efficiency, compatibility	Numerical gates, repeatability, resource records, and interface equivalence
ISO/IEC 25023	Product-quality measurement	Explicit metrics, units, thresholds, samples, and observed values
ISO/IEC/IEEE 29119-2	Test planning, monitoring, execution, completion	Frozen profile, executable checks, canonical report, and completion decision
IEEE 1012	Requirements-based V&V and traceable evidence	Declared domain, independent reference, provenance, and requirement–evidence links

dtype/device and implementation metadata, $\mathcal{S}$ states known scope limits, $\mathcal{Q}$ records the frozen scope and required coverage anchors, and $\mathcal{E}$ contains the requested and observed execution properties, units, and provenance. A result without these fields can be retained for diagnosis but is nonconformant to the selected profile.

The mapping is implemented through requirement identifiers CQ-01–CQ-10. Supplementary Table S5 traces each identifier from a standards concern to a schema field, test action, output artifact, and decision rule. The identifiers are project requirements rather than clauses of the cited standards.

3.1. Conformance profiles and versioning

The **Core** profile requires value and directional-gradient accuracy, batch shape and independence, repeatability, at least eight cases, and nominal, boundary, and heterogeneous-batch anchors. **Extended** requires at least 16 cases and adds perturbation, execution-policy, long-horizon, OOD, dtype/device, unit, and resource evidence. **Application** requires at least 24 cases and adds an application-composition anchor, calibration, and composition evidence. The scope and sampling plan are frozen before candidate execution. Missing mandatory evidence, an unfrozen scope, or insufficient coverage is a nonconformance rather than an omitted score. The current schema is **DFSC-DNC-Conformance-v3**. Migration from v1 or v2 preserves

the original evidence in provenance but marks the record for requalification; it cannot inherit a conformant decision without satisfying the v3 coverage rules.

Interoperability is defined at the record boundary. A consumer selects a profile and submits a record carrying an exact schema identifier. The reference validator accepts v3, explicitly migrates v1 or v2 evidence into a requalification record, and rejects unknown identifiers. Canonical JSON sorts object keys, uses compact separators and ASCII escapes, and is encoded as UTF-8 before a SHA-256 digest is computed. Array order remains significant. The demonstrated claim is limited to the Python API, command-line path, canonical JSON, and the tested PyTorch evidence plugins; JAX, Julia, and independent validators were not evaluated. Semantic changes require a new schema identifier and a declared migration rather than silent interpretation.

3.2. Reliability dimensions

Value fidelity is measured against a declared reference using absolute and relative errors and finite-value rates. For prediction $\hat{y}$ and reference y^* , we use $e_{\text{rel}} = \|\hat{y} - y^*\|_2 / (\|y^*\|_2 + \epsilon)$. Acceptance is based on the mixed criterion

$$\|\hat{y} - y^*\|_2 \leq \tau_{\text{abs}} + \tau_{\text{rel}}\|y^*\|_2, \quad (3)$$

which remains meaningful near zero. For a normalized parameter direction v , the gradient check compares the autodiff directional derivative $g_{\text{AD}} = \nabla_{\theta} P(x; \theta)^{\top} v$ with a central-difference estimate g_{FD} and reports $e_{\text{grad}} = |g_{\text{AD}} - g_{\text{FD}}| / (|g_{\text{FD}}| + \epsilon)$. The finite-difference step is swept around a dtype-scaled nominal value rather than selected from one favorable perturbation; a gradient gate passes only when an agreement region is observed. Differentiability is additionally assessed using finite-gradient rates. Calibration uses noisy observations and reports parameter error, convergence behavior, seed variation, and a matched classical baseline where available. Module reuse evaluates at least two compositions, including a neural module and a batched operator or physics objective. OOD, long-horizon, and batch/device tests expose boundary behavior rather than collapsing it into one score.

The tolerances in Eq. (3) are declared before the audit and are tied to the reference accuracy, dtype, conditioning, and downstream task resolution. A practical calibration starts with reference-only and perturbation runs. The relative tolerance must exceed the combined reference uncertainty and dtype-dependent roundoff after the observed conditioning is taken into

account. The absolute tolerance covers the near-zero regime and should not be smaller than the output resolution that matters to the downstream task. Float32 and float64 receive separate tolerance records. The selected pair is frozen before the candidate implementation is evaluated, and a sensitivity table reports whether modest threshold changes alter the status. For ill-conditioned maps, a condition estimate or perturbation study is recorded so that forward sensitivity is not misclassified as an implementation defect.

Each test dimension retains its measured status. The machine-readable profile decision is binary: **CONFORMANT** means that every mandatory item is present, the scope was frozen, profile coverage requirements are met, and all gates pass on that scope. **NONCONFORMANT** records a failed gate, execution mismatch, coverage failure, migration requiring requalification, or missing evidence. Scope restrictions remain explicit fields and cannot be hidden by an aggregate score. In prose, a conformant record with nonempty restrictions is reported as *conformant with declared scope limits*; this phrase does not introduce a third machine state.

The protocol separates differentiation from numerical accuracy. Automatic differentiation differentiates the executed computational graph, whereas approximation and floating-point errors enter through the evaluator and its dtype. In schematic form,

$$e_{\text{grad}} \lesssim \kappa_{P'}(e_{\text{eval}} + e_{\text{AD}} + e_{\text{ref}}), \quad (4)$$

where e_{eval} denotes the approximation error in the executed primitive, e_{AD} the differentiation-path error, and e_{ref} the error of the independent derivative check. Let $z = (x, \theta)$ and $G(z) = P'(z)$. We interpret the local derivative sensitivity as

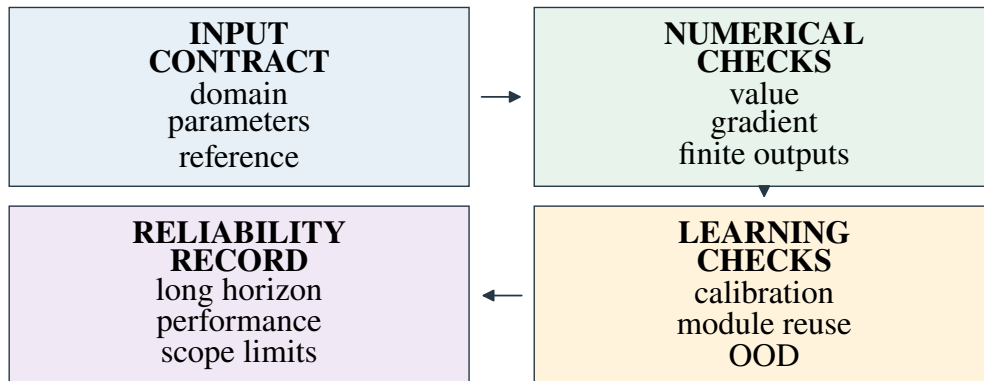
$$\kappa_{P'}(z; s_z) = \lim_{\delta \rightarrow 0^+} \sup_{\|\Delta z\| \leq \delta s_z} \frac{\|G(z + \Delta z) - G(z)\|}{\delta(\|G(z)\| + \epsilon_0)}, \quad (5)$$

with a declared perturbation scale $s_z > 0$ and a fixed output-scale guard $\epsilon_0 > 0$. The inputs and derivative outputs are nondimensionalized, or scaled by recorded physical units, before this diagnostic is evaluated. Accordingly, ϵ_0 has the same normalized units as $\|G(z)\|$; it is fixed from the independent reference resolution before the candidate implementation is run and is stored in the record. This quantity measures the conditioning of the derivative map. For matrix functions it can be evaluated or estimated through Fréchet derivatives (Higham, 2008; Higham and Relton, 2014). Equation (4) is a diagnostic decomposition, not a global error bound.

Two mechanisms require separate checks. Products of step Jacobians can amplify reverse sensitivities in a stiff system even when the forward trajectory is bounded. Nonnormal operators can exhibit transient growth that is not visible from their eigenvalues. We therefore pair perturbation probes with horizon-dependent tests. For a stable p th-order stepper with step size h and N steps, the accumulated local truncation contribution scales as $O(Nh^{p+1})$ before problem-dependent stability amplification. Floating-point accumulation is commonly $O(Nu)$ in unit roundoff u , although cancellation and conditioning may change this behavior (Higham, 2002). The audit record stores dtype, horizon, step size or truncation budget, reference precision, and whether the gradient reference follows the executed path. Float32 and float64 retain separate accuracy records.

4. Reference Software Architecture

The reference implementation is provided in the `P4/dfsc_protocol` package. Existing numerical test actions measure finite outputs, value error, directional-gradient agreement, batch behavior, device locality, and resource use. The new `conformance` module validates the profile, compares requested with observed execution, and checks mandatory evidence. `canonical_json` removes serialization-order ambiguity, while `record_digest` gives a stable content identifier. The same evaluator is exposed through a Python API and a command-line interface. Reports are generated as canonical JSON suitable for continuous-integration gates and exchange between tools.



Every gate is evaluated on the declared domain

backend | dtype | device | batch | seed | reference | scope limit

CONFORMANT / NONCONFORMANT + explicit scope fields

Figure 1: Component conformance workflow. A versioned specification binds the declared domain and requested execution to measured evidence, unmet requirements, operating restrictions, and a canonical machine-readable report.

Every experiment follows the same order. The executable specification and metadata are frozen before sampling. In-domain cases are evaluated against an independent reference, followed by value and directional-gradient gates, calibration and composition tests over fixed seeds, and one-factor OOD or long-horizon extensions. Performance is measured only for configurations with finite outputs that satisfy the numerical gates. The record contains aggregate statistics, the worst observed metric, failed cases, execution properties, and the profile decision.

5. Experimental Setup

The standards-facing evaluation has two parts. First, 40 randomized records test API-CLI equivalence, key-order invariance, evidence-preserving legacy migration, and mandatory requalification after migration. Ten mutation operators target truncation evidence, gradient connectivity, dtype, batch isolation, device locality, OOD scope, units, provenance, scope freezing, and minimum profile coverage. Each operator is applied 20 times, and 40 unmodified records estimate false rejection. Four predeclared strategies are replayed

Table 3: Required evidence and acceptance logic. Thresholds are declared per backend and domain; the specification standardizes fields and decisions, not a universal numerical value.

Gate	Required observable	Acceptance record
Value	Mixed absolute/relative error; finite-output rate	Threshold, reference, dtype, domain
Gradient	Directional error over a step-size sweep; finite-gradient rate	Stable agreement region and failed directions
Calibration	Parameter error, loss, convergence, seed variation	Optimizer, bounds, initialization, noise
Reuse	At least two differentiable compositions	Shape/device checks and task metric
Stress	OOD and long-horizon error trajectories	Tested extension and first failed regime
Performance	Synchronized latency, throughput, memory	Hardware, dtype, batch, warmup, repeats

on these same records and mutations: envelope plus value/gradient checks; numerical properties, which add batch isolation, repeatability, OOD, and horizon evidence; execution-aware checks, which add dtype, device, and unit evidence; and the complete record, which also enforces provenance, frozen scope, and profile coverage. Fault class is the primary comparison unit. These deliberately minimal strategies isolate evidence increments and are not claimed to represent every implementation of unit, property-based, or metamorphic testing. Second, numerical experiments provide the measured evidence consumed by the profiles. They combine controlled low-dimensional components with a direct two-dimensional periodic heat-equation component. The four initial backends are MLSL, matrix-exponential action, fixed-step linear RK4, and Logistic RK4; the two-dimensional extension uses a differentiable Fourier propagator. Unless otherwise stated, multi-seed experiments use five seeds. The common neural OOD task uses three seeds, 512 training tasks, 256 test tasks, two target times (1.0 and 1.5), and two observation-noise levels (0.003 and 0.01) on an RTX 5070 Laptop GPU. The PDE test uses five grids from 16^2 to 256^2 , five seeds, and three horizons; a finite trigonometric expansion supplies a closed-form reference independent of the FFT

implementation.

The external-subject pilot replays four cumulative strategies on three installed PyTorch interfaces: scalar special-function differentiation, matrix-exponential action, and a non-contiguous CUDA contraction. Eight behavior-fault families, ten seeds, and three interfaces give 240 injected trials; 30 unmodified cases are clean controls. Subject-fault pairs, rather than repeated seeds, form the 24 heterogeneity clusters. A separate historical-defect intake freezes upstream provenance and observations for two PyTorch reports and SciPy #8906. For the latter, 32 surrogate variants first compare a padded example that masks the reported hard-coded 1x1 band-row choice with a mathematically equivalent conventional representation. Unchanged runners then compare official buggy wheels with installed fixed releases for this case, PyTorch #80770, and SciPy #15620. The last case tests a dtype relation: integer and floating inputs to `resample_poly` must represent the same nonzero signal. Interpreter, wheel, source or fix-commit, command, output, and runner hashes are retained.

6. Results

The reported metrics have backend-specific meanings and scales. Comparative plots are interpreted against each backend’s declared test and tolerance; timing values are descriptive when execution stacks differ. Table 5 summarizes how the detailed experiments populate the three cumulative profiles.

6.1. Interface interoperability and fault detection

All 40 randomized trials produced byte-identical canonical records through the Python API and command-line interface, and key reordering did not change the digest. Legacy migration preserved the submitted evidence in all 40 trials, while every migrated record was correctly marked for requalification rather than inheriting its earlier decision. The mutation study detected 200 of 200 injected faults. The Wilson 95% interval for pooled detection is $[0.981, 1.000]$. Each of the ten mutation categories was detected in 20 of 20 trials, with a category-wise lower bound of 0.839. None of 40 clean records was rejected; the Wilson interval for false rejection is $[0, 0.088]$. These intervals quantify finite repeated tests under the selected mutation model. They neither estimate field-defect prevalence nor establish detection of arbitrary faults.

The paired strategy comparison localizes the gain. Table 4 shows a monotone increase from 3 to 10 detected fault classes as execution, provenance, and scope evidence are added. All four strategies accepted the 40 clean records, so the additional coverage did not trade against specificity on this test set.

Table 4: Paired comparison on the same ten fault classes (20 mutations per class) and 40 clean records.

Strategy	Classes	Instances	Clean rejects
Envelope + value/gradient	3/10	60/200	0/40
Numerical-property suite	5/10	100/200	0/40
Execution-aware suite	8/10	160/200	0/40
Executable evidence record	10/10	200/200	0/40

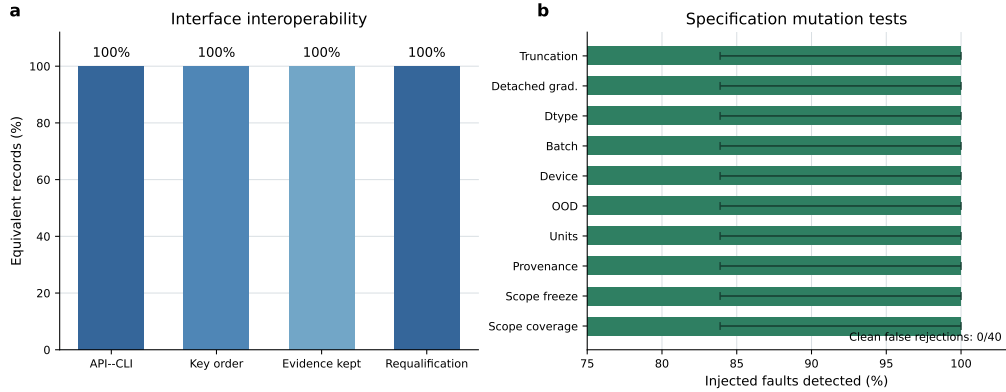


Figure 2: Validation of the reference implementation. Panel (a) reports API-CLI equivalence, key-order invariance, evidence preservation during migration, and mandatory requalification over 40 randomized trials. Panel (b) reports ten mutation categories, 20 trials per category; whiskers show Wilson 95% intervals. The clean-record test produced 0/40 false rejections.

6.2. External interfaces and historical-defect-derived strategy

On the 240 external-subject injected trials, detection increased from 25.00% for a single-point test to 73.75% after directional-gradient evidence, 99.17% for the numerical-property suite, and 100% for complete execution evidence. All strategies accepted the 30 clean controls. At the 24 subject-fault cluster level, the complete strategy detected every cluster; successive

layers added 3, 7, 7, and 1 fully detected clusters without losing a previously detected cluster. Leave-one-interface-out detection remained 1.000 for the complete suite. These are controlled behavior mutations within one vendor project, not field-defect prevalence estimates.

The SciPy #8906-derived comparison answers RQ3 on one historical defect family. The weak padded example detected 0/32 paired variants because its chosen representation happened to place the coefficient in the faulty hard-coded row. Requiring equal solutions for the padded and conventional representations detected 32/32 variants; current SciPy 1.18.0 satisfied the property in 32/32 controls. For PyTorch #80770, the identical runner (SHA-256 `fdb828...014`) returned a zero `xlogy(0,2)` derivative in the official 1.11.0+cpu wheel and log 2 with zero stored error in current PyTorch 2.11.0+cu128. For SciPy #8906, a second unchanged runner (SHA-256 `ce901...bc8`) made the conventional 1x1 representation raise the reported `IndexError` in the official 1.14.1 wheel while the padded representation solved exactly; current 1.18.0 solved both exactly. For SciPy #15620, a third runner (SHA-256 `0a7a72...f79`) produced all-zero outputs for both integer dtypes in official 1.14.1 although the floating reference reached 3.001552; current 1.18.0 matched the float64 reference with zero maximum error. All six role checks passed, completing three historical buggy/fixed environment pairs across two projects.

A matched cross-project subset applied seven output/execution fault families to `numpy.linalg.solve`, `scipy.linalg.expm`, and `torch.lgamma`. On 252 injected trials, detection increased from 28.57% for a single point to 57.14% for full-batch values, 85.71% for numerical properties, and 100% for execution evidence. All 36 clean controls passed, and the complete strategy detected all 21 project–subject–fault clusters. This closes the controlled three-project count, although the NumPy and SciPy arms do not test autodiff and none of these wrappers is a field defect.

6.3. Numerical gate coverage

The registry contains four backend records and six required dimensions: value, gradient, calibration, module reuse, OOD, and long horizon. We additionally reran the MLSL backend through the P1 public factory API using the stable hybrid configuration. On the declared one-dimensional Dirichlet, negative-real spectral regime, the maximum relative value error against an 80-digit reference series was 1.32×10^{-12} ; alpha and beta gradient errors

against central differences were 2.22×10^{-9} and 5.00×10^{-10} . Joint calibration recovered $(\alpha, \beta) = (0.78003, 1.80007)$ from $(0.90, 2.10)$ initialization. MLSL meets all six gates on this regime and retains the stated operating restrictions. All four records are *conformant with declared scope limits*; their machine-readable status is **CONFORMANT**, and the restrictions remain part of each decision record.

Table 5: Profile evidence and retained operating restrictions. Profiles are cumulative; the table points to the evidence classes summarized below rather than replacing the measured results.

Backend	Core evidence	Extended evidence	Application evidence and recorded restriction
MLSL	Value and α/β gradient gates	OOD and horizon probes	Joint calibration and hybrid reuse; one-dimensional Dirichlet problems with negative-real spectral arguments
Matrix exponential action	Value and directional-gradient gates	Long-horizon probes	Two-parameter calibration; neural composition gives a negative task result; stable 2×2 parameter box
Linear RK4 step	Value and directional-gradient gates	Step-size OOD and horizon probes	Calibration and reuse; stable linear systems at recorded step sizes
Logistic RK4 step	Value and directional-gradient gates	Step-size OOD and horizon probes	Calibration and neural composition; tested scalar parameter and time ranges

The matrix action remains near reference precision over the tested horizon. Both RK4 implementations exhibit the expected increase in error outside their declared step-size ranges, with the linear system showing the larger OOD error budget. Figure 3 shows the seed-wise distributions; the exact stress ranges and error values are reported in Supplementary Table S2.

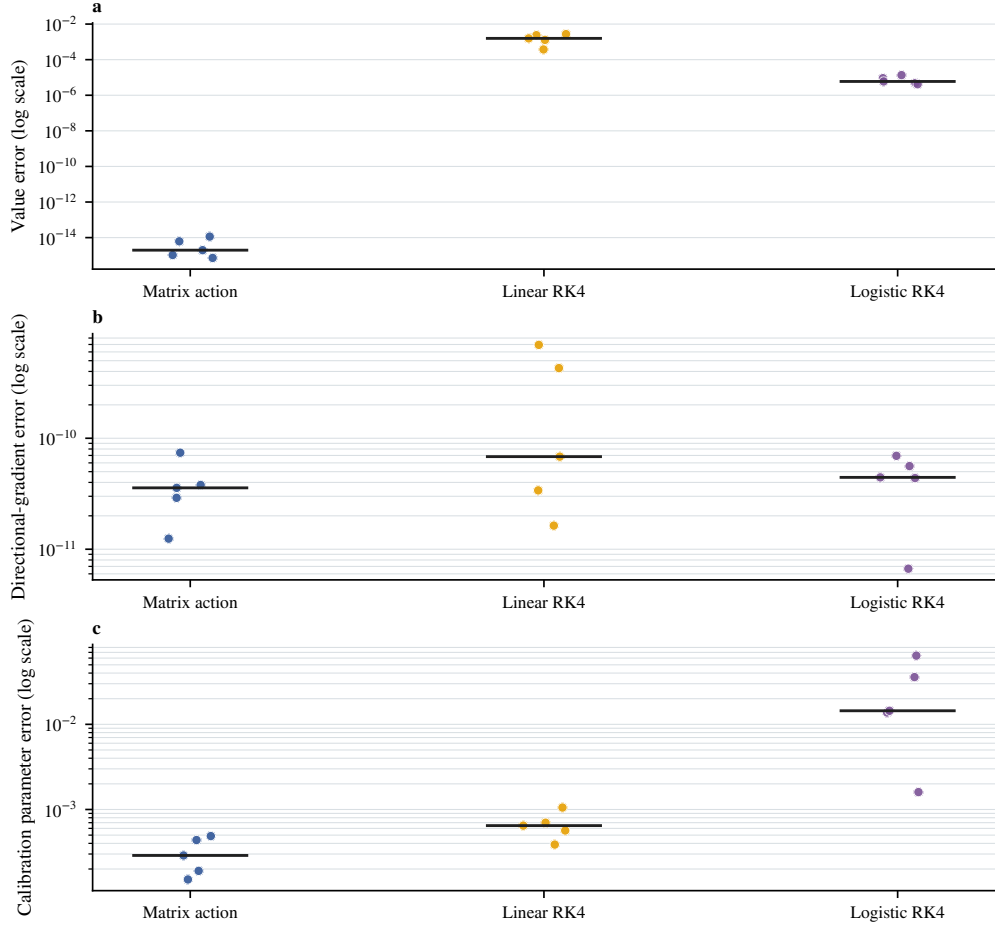


Figure 3: Backend-specific reliability signatures over five fixed seeds. Each point is one seed and the horizontal segment is the median. The value-error panel uses the declared stress metric for each backend (long-horizon maximum error for the matrix action and OOD maximum error for both RK4 steps); the remaining panels use the same directional-gradient and calibration definitions. All axes are logarithmic.

The same records also cover repeated simulation calls. We evaluated one fixed stable linear system $y'(t) = Ay(t)$ over horizons $t = 1, 2, 4, 8, 12$. The exact matrix exponential action and fixed-step RK4 used the same initial states and final times over five seeds. The matrix action remains at reference precision, while RK4 errors decrease with step size under the declared stable dynamics (Figure 4).

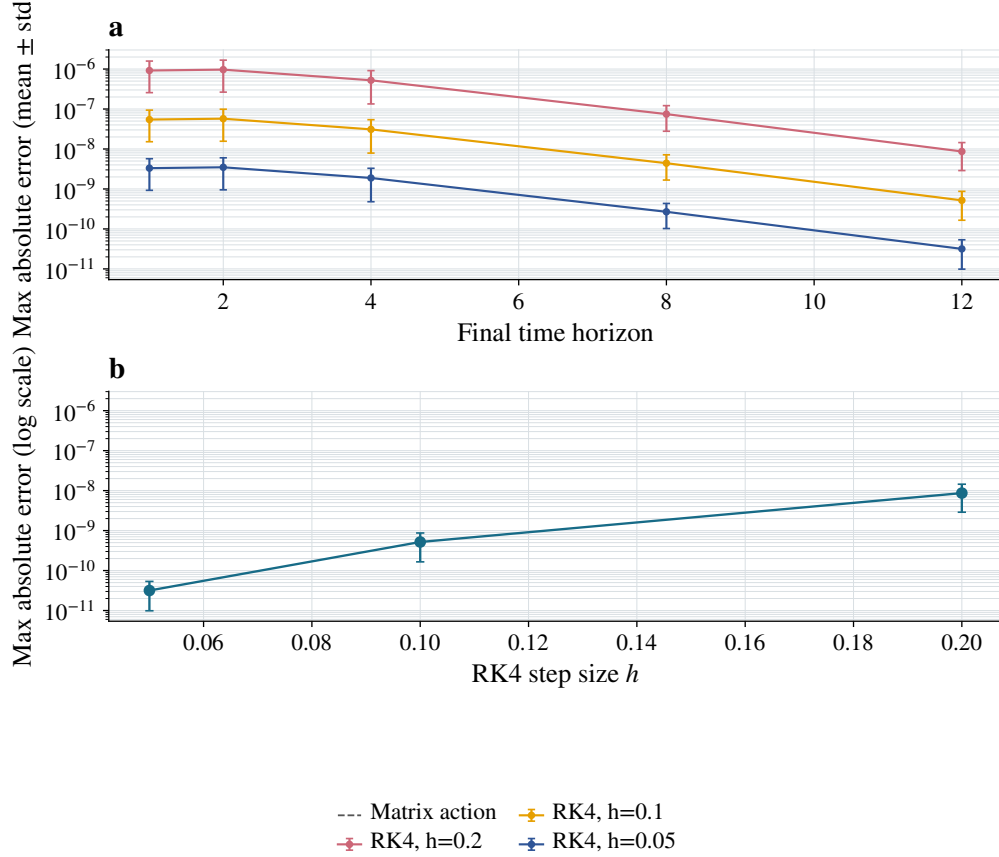


Figure 4: Trajectory and step-size evidence on a fixed stable 2×2 linear system. The left panel reports error over increasing final horizons; the right panel reports RK4 step-size sensitivity at $t = 12$. Error bars are standard deviations over five initial-state seeds.

As an optimizer-level check, the same stable 2×2 matrix-exponential action was calibrated from noisy observations with Adam and L-BFGS-B (Byrd et al., 1995) over five fixed seeds. Both methods reached comparable parameter error and final loss under matched bounds and initialization. Fig-

ure 5 shows the paired recoveries; per-seed values, work counts, and execution times are reported in Supplementary Table S1.

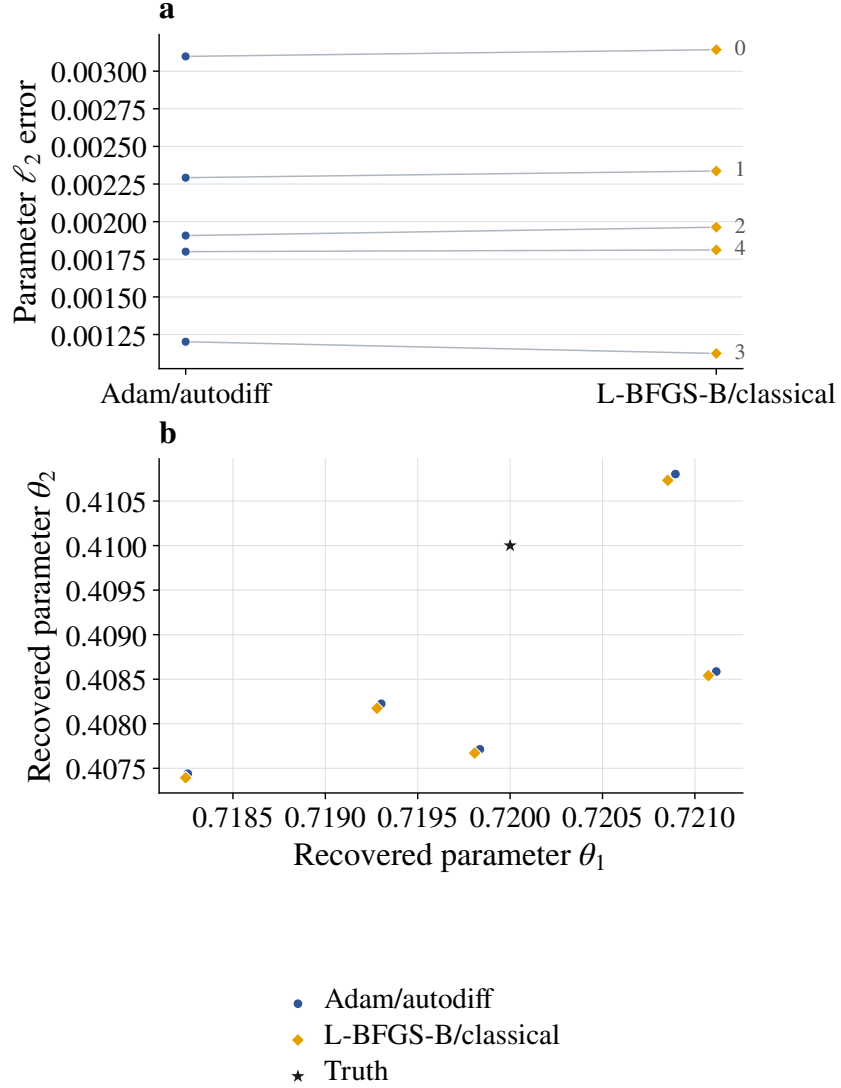


Figure 5: Seed-wise reproducibility of the matched two-parameter calibration. The left panel pairs parameter errors obtained from the same noisy observation for Adam/autodiff and L-BFGS-B/classical optimization. The right panel shows the recovered parameter pairs relative to the ground truth; connecting segments identify matched seeds.

6.4. Task-level transfer and OOD evidence

We additionally evaluated the structured primitive on two provenance-aware public datasets. For the GeoTES pilot-scale thermocouple histories (Zenodo DOI 10.5281/zenodo.18979098), the first measured heating/cooling cycle was used for identification and the second cycle for transfer. Across three seeds, the DFSC-plus-residual model obtained a cycle-2 relative error of 0.3619 ± 0.0085 , compared with 0.3632 ± 0.0071 for a pure MLP, using 750 rather than 1251 parameters. For the heated-steam temperature profiles (Zenodo DOI 10.5281/zenodo.15064388), held-out-condition RMSE was 43.33 ± 0.56 K for direct DFSC, 16.03 ± 0.14 K for the hybrid, and 13.25 ± 1.45 K for the pure MLP. The latter result is retained as a negative task-level finding: a reliable primitive does not imply unconditional predictive superiority. Within the **Application** profile, these datasets test whether task evidence and operating restrictions are carried into the same record as component-level conformance. The negative result is operationally useful because it prevents a conformant numerical component from being reported as a superior predictive model without matched task evidence.

To separate observation density from extrapolation-horizon effects, we also repeated the GeoTES experiment with uniform subsampling over the full first cycle while retaining the second cycle as a fixed holdout. At 40% coverage, the hybrid reached 0.3607 ± 0.0082 versus 0.3657 ± 0.0084 for the pure MLP, again with 750 versus 1251 parameters. The hybrid did not improve the MLP at 20%, 60%, or 100% coverage. Its benefit is confined to the moderate-data setting observed here; Figure 6 reports both public-data comparisons.

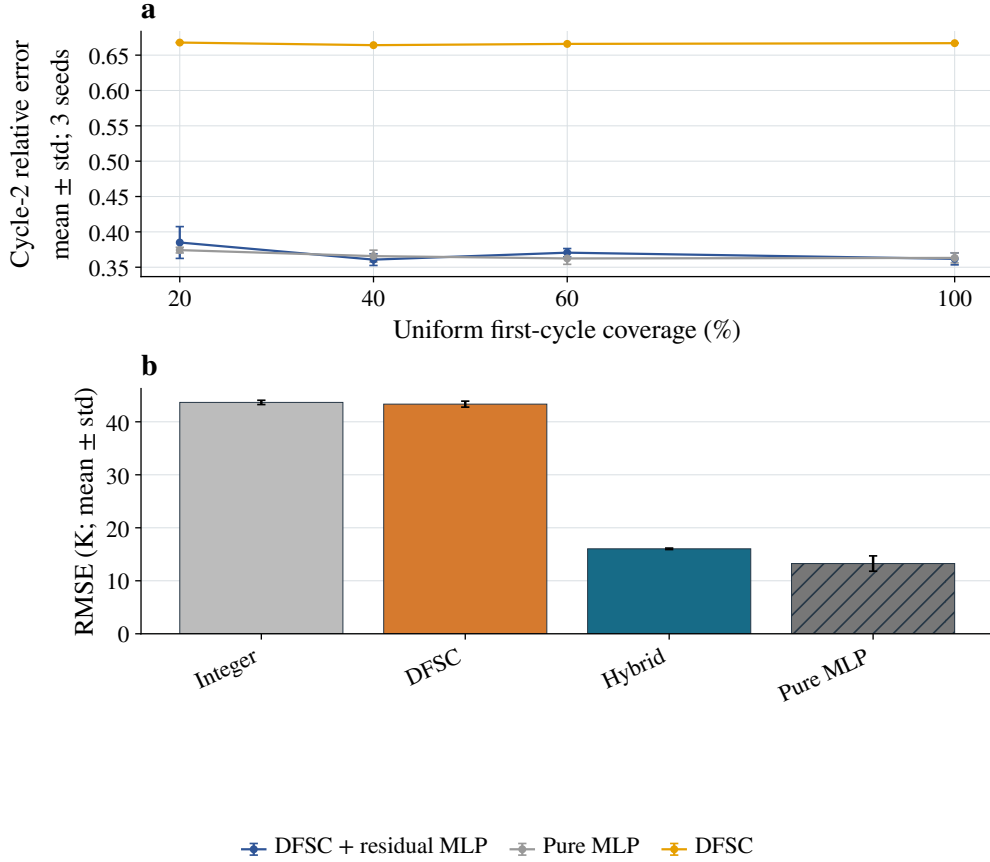


Figure 6: Public-data transfer evidence. The GeoTES panel reports cross-cycle error under uniform first-cycle coverage; the heated-steam panel reports held-out-condition RMSE. Error bars are standard deviations over three seeds. The negative heated-steam comparison is retained to show that primitive reliability does not imply universal task-level superiority.

Module-level OOD evidence.. In the common neural task, the Logistic primitive improves OOD RMSE relative to a pure MLP at both target times and both noise levels: for example, at $t = 1.5$ and noise 0.003, the primitive gives 0.2340 ± 0.0323 versus 0.2713 ± 0.0096 for the MLP. The matrix primitive does not improve this task: at the same setting it gives 0.1998 ± 0.0272 versus 0.1638 ± 0.0097 . Figure 7 shows the full matched comparison. Training with a primitive is also slower in this implementation. The matched outcomes show that composition benefit depends on the task and the identifiable latent

structure.

The negative matrix result does not originate from failure of the matrix exponential evaluator. In this task, the structured model must infer a sample-dependent 2×2 generator A from sparse trajectory snapshots and then predict through $\exp(tA)y_0$. The MLP predicts the target state directly. The structured route adds a latent-parameter identification problem to state prediction. For a diagonalizable generator $A = V\Lambda V^{-1}$,

$$\|\exp(tA)\| \leq \kappa(V) \exp\left(t \max_i \operatorname{Re} \lambda_i\right), \quad (6)$$

so weakly damped modes and an ill-conditioned eigenvector basis can amplify errors in an inferred generator; nonnormal transient growth can make eigenvalues alone insufficient. We tested this explanation over 12 seed–time–noise configurations and correlated sample-wise OOD error with inferred-generator diagnostics. The mean Pearson correlation between log eigenvector condition number and log state error was -0.044 , and the mean Spearman correlation was -0.050 . Eigenvector conditioning alone does not explain the failure. Non-normality and latent-parameter error had modest positive mean correlations of 0.305 and 0.290 , respectively.

We also tested a prospective local-observability screen. For the three observed states, we formed the Jacobian of $[\exp(t_1 A)y_0; \exp(t_2 A)y_0; \exp(t_3 A)y_0]$ with respect to the four entries of A . Neither its smallest singular value nor its condition number supplied the expected monotone warning: the mean Pearson and Spearman correlations between log smallest singular value and log state error were 0.46 and 0.50 , while the mean Pearson correlation for the log Jacobian condition number was -0.19 . A single scalar threshold is therefore unsupported by these data. The audit records local observability, perturbation sensitivity, nonnormality, and parameter-fit diagnostics together; an adverse combination triggers targeted perturbation and matched-baseline tests rather than an automatic rejection. Passing value and gradient gates establishes evaluator reliability on the declared domain, but it does not resolve a poorly identified latent inverse problem.

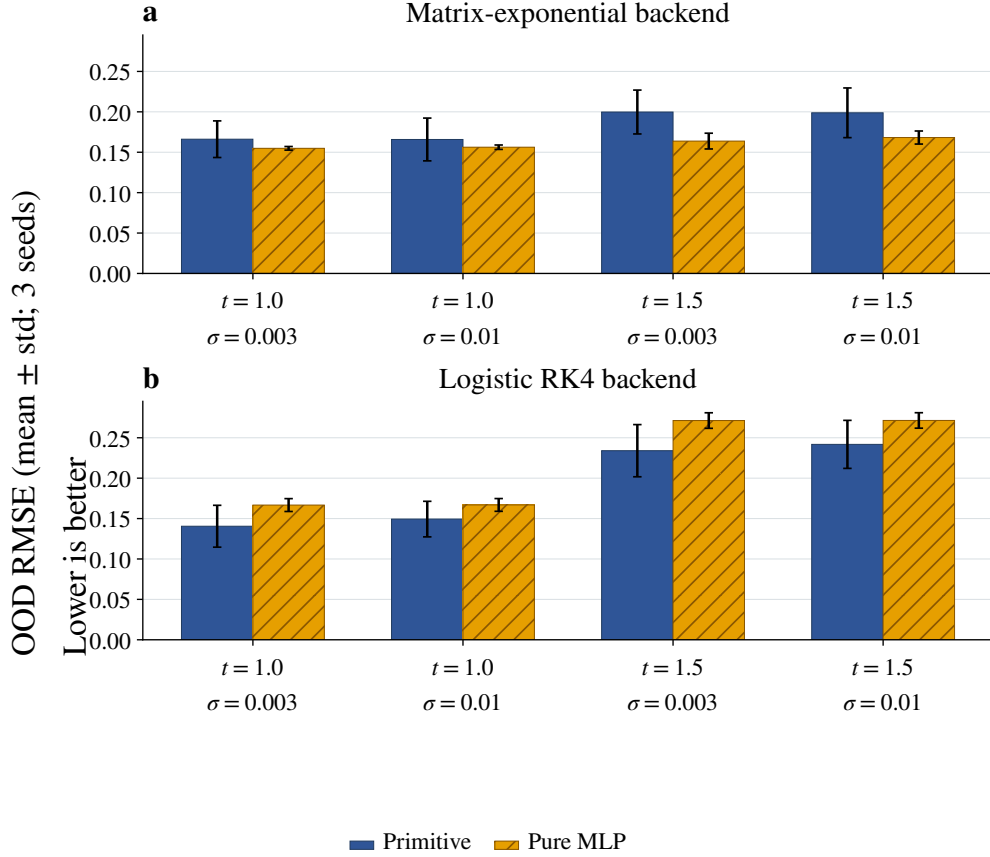


Figure 7: Matched module-level OOD task. The two panels separate the matrix and Logistic backends; bars show means and whiskers show standard deviations over three seeds. Lower RMSE is better. The Logistic primitive improves the pure MLP control in the tested settings, whereas the matrix primitive does not uniformly improve it.

6.5. Execution and system-level audit

On the RTX 5070 Laptop GPU, all tested backends produced finite outputs for batch sizes 1, 64, 256, and 1024 in float64. Throughput increased with batch size (Figure 8). The profile records warmup count, synchronized timing, peak memory, dtype, and output shape so that future backends can be evaluated with the same reporting fields.

We also repeated a batch-1024 matrix-exponential action and a 256^2 periodic heat propagation in float32 and float64. Float32 value errors were of order 10^{-7} , whereas float64 reduced them to order 10^{-15} . Incremental

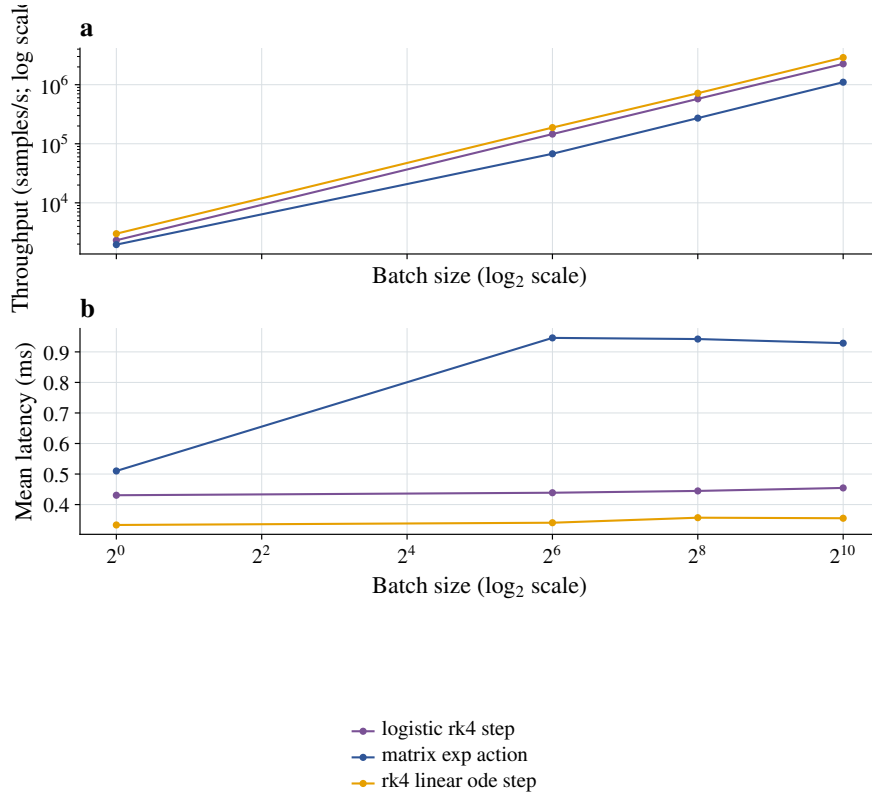


Figure 8: Batch/device profile on the RTX 5070 Laptop GPU. The left panel reports throughput and the right panel reports mean latency for float64 execution over 50 iterations.

CUDA memory was approximately halved in float32, but synchronized latency changed by only about 3% for these launch-dominated workloads. Figure 9 summarizes the trade-off; exact value and gradient errors, latency, and memory measurements are listed in Supplementary Tables S3–S4. Gate tolerances are calibrated separately for each dtype and reference accuracy.

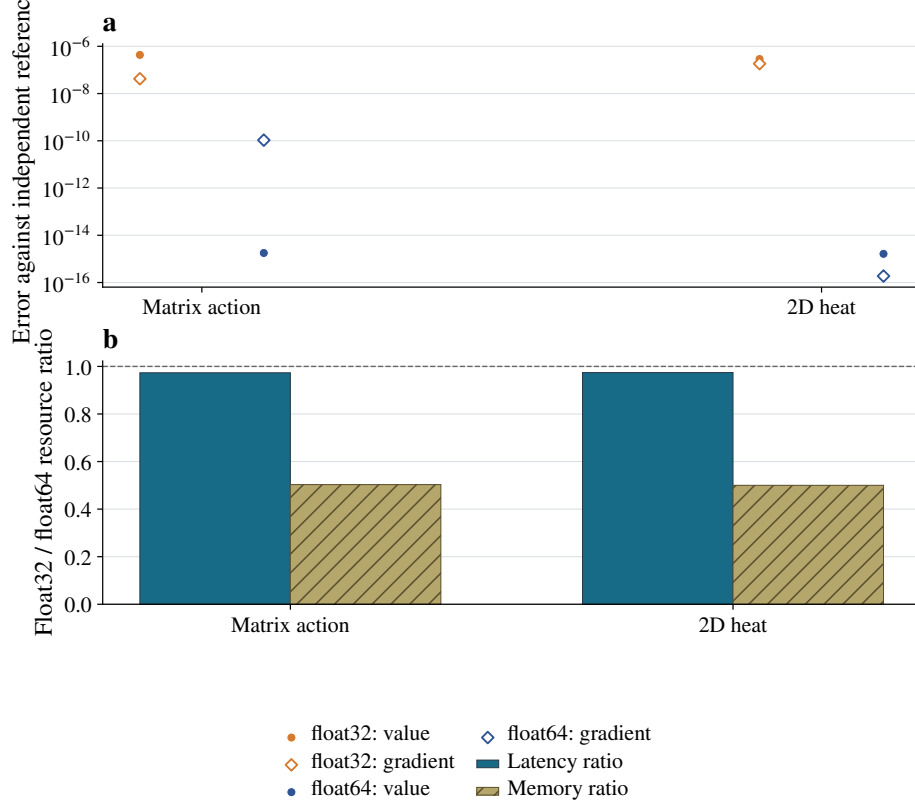


Figure 9: Dtype-specific accuracy and resource trade-off on the RTX 5070 Laptop GPU. Panel (a) compares value and parameter-gradient errors against independent references; panel (b) reports float32-to-float64 latency and incremental-memory ratios.

Direct two-dimensional PDE audit. We next applied the same protocol to the periodic heat equation $u_t = \kappa \Delta u$ on $[0, 2\pi]^2$. The primitive evaluates the Fourier multiplier $\exp[-\kappa(k_x^2 + k_y^2)t]$, while a finite analytic mode expansion provides an implementation-independent reference for both the field and its κ derivative. Across 75 grid-seed-horizon configurations, from 16^2 to 256^2 states and from $t = 0.1$ to 1.0 , the worst maximum absolute field error was 6.99×10^{-15} and the worst relative energy-gradient error was 4.75×10^{-16} . All outputs and gradients were finite. On the RTX 5070 Laptop GPU, median forward latency remained between 0.369 and 0.397 ms for the tested single-field calls, while peak incremental memory increased from 0.011 to 2.50 MiB. The nearly flat timing at these sizes is launch-dominated and must not be extrapolated as asymptotic FFT scaling; the memory trend

follows the increasing number of spatial degrees of freedom (Figure 10).

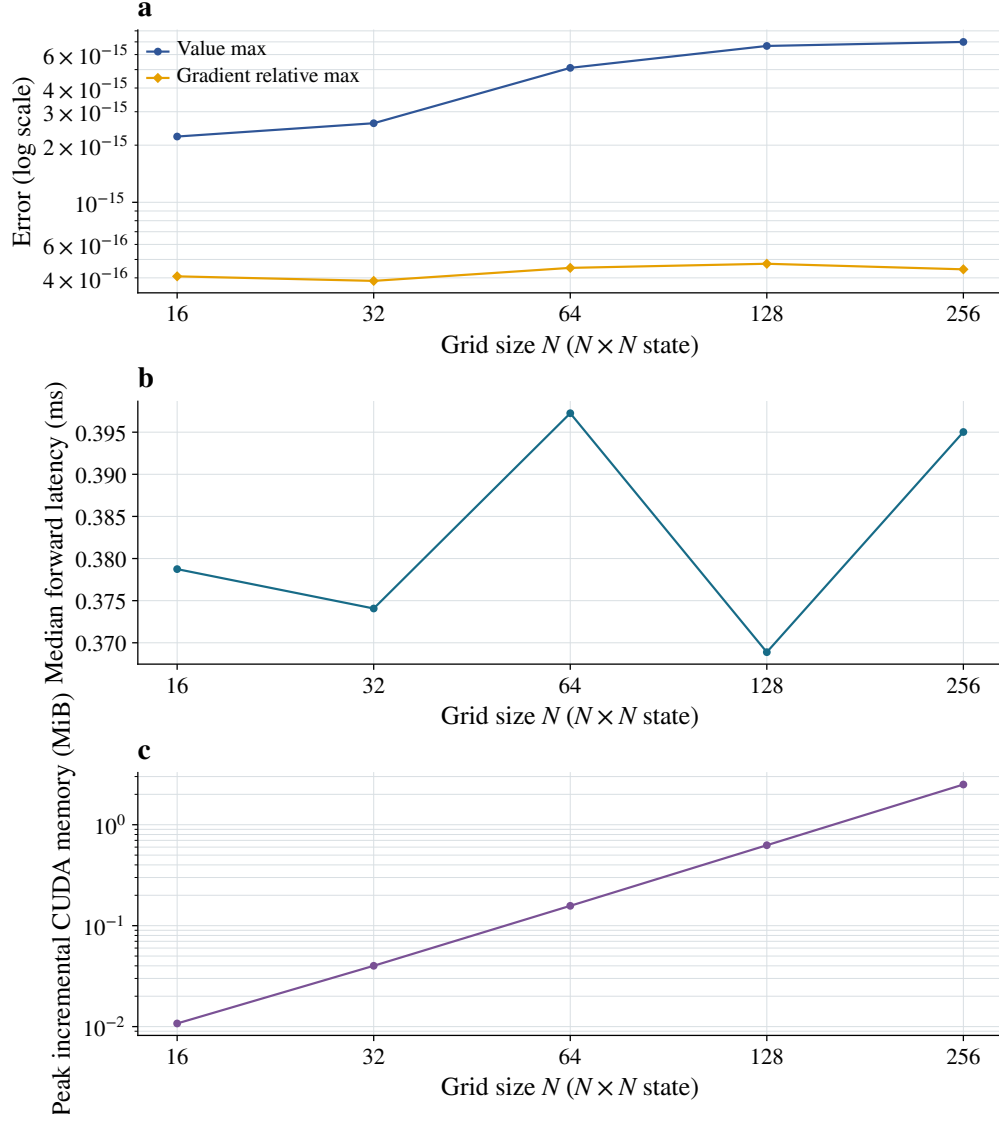


Figure 10: Direct two-dimensional periodic heat-equation audit. The left panel reports worst field and parameter-gradient errors over five seeds and three horizons; the center and right panels report synchronized median forward latency and peak incremental CUDA memory. The closed-form mode expansion is independent of the FFT implementation.

7. Discussion

The standards mapping gives the proposal a defined role between numerical analysis and software test processes. ISO/IEC 25010 and 25023 motivate the quality characteristics and measurable fields; ISO/IEC/IEEE 29119-2 motivates the frozen test process and completion decision; IEEE 1012 motivates traceability from requirements to independent evidence. Together, these mappings provide traceable provenance for the component-level specification and its executable tests.

The mutation results show that the current rules detected all ten fault classes represented in the experiment, including an unfrozen scope and insufficient profile coverage. The paired strategy comparison adds a more specific conclusion: the envelope plus value/gradient strategy leaves batch, OOD, execution-policy, and frozen-scope defects invisible, whereas each added evidence layer closes declared fault classes without increasing false rejection on the clean controls. This is a comparative result for the frozen catalogue, not a claim that the complete record subsumes all testing methods. The Wilson intervals describe repeated tests of this implementation and mutation model; they do not estimate field-defect prevalence or prove completeness. Further mutation operators should cover malformed units, adversarial provenance, partial device fallback, stochastic nondeterminism, and cross-version semantic drift. Independent implementations are also needed to test whether the schema is sufficiently precise without relying on this reference code.

The external-subject ordering and complete PyTorch #80770 and SciPy #8906/#15620 pairs improve realism without closing external validity. The matched subset reaches three independently maintained projects, but each project contributes only one interface to that subset and the NumPy/SciPy arms omit autodiff. Repeated seeds and representations remain within-family trials, not independent projects or defects. Three real families now cover a wrong derivative, an indexing exception, and silent dtype-dependent output, removing the earlier two-family limitation. They still span only two projects, however, so a JSS-level field-effectiveness claim requires broader historical replication.

The trajectory experiment connects the component tests to repeated simulation. Value and gradient audits characterize one call; repeated application additionally depends on the time horizon and step-size error budget. Figure 4 quantifies this dependence for the controlled propagators studied here. Nonlinear, stiff, and complex-geometry systems require corresponding

trajectory tests before the same qualification state can be assigned. The experiments cover controlled low-dimensional components, one structured two-dimensional periodic PDE, two public-data transfer tests, and PyTorch execution. They show that the protocol can be applied to different numerical structures. They do not cover variable- or distributed-order operators, nonlinear or stiff PDEs, complex geometries, or unstructured meshes. Extending the protocol to those cases requires four additions: mesh and geometry metadata in the operating domain; an independent control based on manufactured solutions, mesh refinement, or a distinct discretization; convergence records for nonlinear solves and preconditioners; and horizon-dependent value, gradient, and conservation gates. These additions define new qualification studies rather than extending the present **CONFORMANT** decisions by assumption. JAX and Julia also require separate device and autodiff validation. The 2×2 matrix experiment exposes an interaction between weak identifiability and nonnormality, but does not quantify its growth in larger systems.

For higher-dimensional simulation, the audit budget must scale with the discretization. Let n be the state dimension, B the batch size, K the number of value/reference cases, and q the number of directional-gradient probes. If one primitive and reference evaluation cost $C_P(n)$ and $C_R(n)$, respectively, the leading work is approximately $K[C_P(n) + C_R(n)] + q[C_{P,AD}(n) + 2C_R(n)]$.

Directional probes avoid materializing a full parameter Jacobian. Live-state storage is at least $O(Bn)$, while reverse-mode graph storage depends on the primitive. The executable specification records chunk size, sparse structure, checkpointing, and peak device memory.

The independent reference may dominate the cost. A sparse finite-element primitive can use Krylov actions, while its control may require mesh refinement, a higher-order element, or a distinct discretization. Nonlinear and stiff problems add converged nonlinear solves, Jacobian or preconditioner construction, and trajectory-level references. Mesh-free primitives add dependence on the number of collocation points and residual derivative order.

When no closed form exists, manufactured solutions, conservation balances, and refinement studies can serve as controls. A reference built from the same discretization must be marked as non-independent. The periodic heat experiment applies this design to 65,536 spatial states and records grid size, value error, gradient error, synchronized latency, and peak memory. Navier–Stokes, stiff multiphysics, nonperiodic boundaries, and unstructured finite-element meshes remain separate qualification studies.

7.1. Transfer to other component classes and standardization path

The transferable part of the proposal is the record boundary, not a universal set of numerical gates. Database kernels, simulation modules, and AI inference components could reuse schema identity, version migration, provenance, requested-versus-observed execution fields, canonical serialization, and qualification outcomes. Each class would still need its own evidence plugin, independent control, tolerances, mutation operators, and operating-domain metadata. The experiments in this paper qualify the four studied numerical backends only; they do not establish conformance criteria for those other component classes.

Progress from a research specification to a consensus standard would require normative field definitions, public conformance examples, at least one independent implementation, and round-robin tests across organizations. A standards process would also need governance for controlled vocabularies, version deprecation, security and provenance requirements, and procedures for resolving incompatible interpretations. The released schema and artifacts provide executable inputs for such evaluation and interoperability studies.

7.2. Component selection in SciML workflows

A structured primitive is a reasonable choice when its parameters are identifiable from the available observations, an independent reference exercises the declared domain, and its value and gradient gates remain stable over the intended horizon. Task-level benefit must still be compared with an unstructured control.

Sparse or noisy observations can make the latent inverse problem underdetermined. Nonnormal or stiff dynamics can amplify transient errors and reverse-mode Jacobian products. A same-path reference provides weaker evidence than an independently constructed one. In these cases the qualification record remains **CONFORMANT** only when every mandatory gate passes on the restricted domain; otherwise it is **NONCONFORMANT**. Perturbation and identifiability diagnostics, together with a residual hybrid or direct learned baseline, explain the decision. A failed gate is a qualification result, not a software exception.

8. Reproducibility and Software Availability

Experiment scripts, JSON results, schema validators, package tests, and the paper-data builder are available under P4 in the public DFSC repository.

The conformance, synthetic strategy, external-subject, and historical-defect-derived records are stored under `P4/results/`; the API and CLI paths share only the evaluator, not the serialization route. The repository preserves the implementation, inputs, provenance, and claim boundaries needed for reproduction.

9. Conclusion

We proposed and implemented an executable testing and qualification method for differentiable numerical components. On a paired ten-class catalogue, fault-class coverage increased from 3/10 for envelope plus value/gradient checks to 5/10 for numerical properties, 8/10 for execution-aware checks, and 10/10 for the complete record. The same ordering reached 25.00%, 73.75%, 99.17%, and 100% detection on 240 external PyTorch behavior-fault trials. A matched NumPy/SciPy/PyTorch subset reached 28.57%, 57.14%, 85.71%, and 100% on 252 trials, closing the controlled three-project gate. Representation equivalence derived from SciPy #8906 detected 32/32 variants missed by a weak correct example. Unchanged runners distinguish official buggy and current fixed behavior for PyTorch #80770 and SciPy #8906/#15620, completing three historical pairs across two projects. The method therefore connects incremental evidence to external interfaces and real release-level defects while remaining a falsifiable component-level argument rather than a universal reliability guarantee.

Funding

This work was supported by the State Key Laboratory of Ocean Engineering, Shanghai Jiao Tong University (Grant No. GKZD010089), and the State Key Laboratory of Acoustics, Chinese Academy of Sciences (Grant No. SKLA202406). The funders had no role in the study design; collection, analysis, and interpretation of data; writing of the manuscript; or decision to submit the article for publication.

Data availability

The conformance implementation, numerical experiments, generated JSON reports, tests, paper-data builder, and figure-generation workflow are publicly available in the tagged P4 directory of the DFSC repository at github.com/hzhooning-art/DFSC/tree/p4-jss-submission-v1.2/P4.

The standards-facing interoperability and mutation-test data are stored in `P4/results/p4_csi_conformance_validation.json`. Public source datasets are cited with repository DOIs in the corresponding experiments. The archival software DOI will be added after publication of the accompanying Zenodo record.

CRedit authorship contribution statement

Ning Hu: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Visualization, Writing - original draft, Writing - review and editing. Haitao Duan: Methodology, Software, Validation, Investigation, Writing - review and editing. Shuqun Li: Validation, Investigation, Writing - review and editing. Chuyang Hu: Validation, Investigation, Writing - review and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used OpenAI Codex to support language editing, organization of manuscript drafts, consistency checks, and code and documentation review. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication. The tool was not used to generate or fabricate experimental data.

Acknowledgements

This research used computational resources of Hangzhou Dianzi University.

References

- Al-Mohy, A.H., Higham, N.J., 2009. Computing the frechet derivative of the matrix exponential, with an application to condition number estimation. *SIAM J. Matrix Anal. Appl.* 30, 1639–1657. doi:10.1137/080716426.
- Al-Mohy, A.H., Higham, N.J., 2011. Computing the action of the matrix exponential, with an application to exponential integrators. *SIAM J. Sci. Comput.* 33, 488–511. doi:10.1137/100788860.
- Baydin, A.G., Pearlmutter, B.A., Radul, A.A., Siskind, J.M., 2018. Automatic differentiation in machine learning: A survey. *J. Mach. Learn. Res.* 18, 1–43. URL: <https://jmlr.org/papers/v18/17-468.html>.
- Byrd, R.H., Lu, P., Nocedal, J., Zhu, C., 1995. A limited memory algorithm for bound constrained optimization. *SIAM J. Sci. Comput.* 16, 1190–1208. doi:10.1137/0916069.
- Chen, R.T.Q., Rubanova, Y., Bettencourt, J., Duvenaud, D.K., 2018. Neural ordinary differential equations, in: *Advances in Neural Information Processing Systems*, pp. 6571–6583. URL: <https://proceedings.neurips.cc/paper/2018/hash/69386f6bb1dfed68692a24c8686939b9-Abstract.html>, arXiv:1806.07366.
- Garrappa, R., 2015. Numerical evaluation of two and three parameter mittag-leffler functions. *SIAM J. Numer. Anal.* 53, 1350–1369. doi:10.1137/140971191.
- Gorenflo, R., Kilbas, A.A., Mainardi, F., Rogosin, S.V., 2014. *Mittag-Leffler Functions, Related Topics and Applications*. Springer, Berlin. doi:10.1007/978-3-662-43930-2.
- Higham, N.J., 2002. *Accuracy and Stability of Numerical Algorithms*. 2 ed., Society for Industrial and Applied Mathematics, Philadelphia. doi:10.1137/1.9780898718027.
- Higham, N.J., 2008. *Functions of Matrices: Theory and Computation*. Society for Industrial and Applied Mathematics, Philadelphia. doi:10.1137/1.9780898717778.

- Higham, N.J., Relton, S.D., 2014. Estimating the condition number of the frechet derivative of a matrix function. *SIAM J. Sci. Comput.* 36, C617–C634. doi:10.1137/130950082.
- Holl, P., Thuerey, N., Koltun, V., 2020. Learning to control PDEs with differentiable physics, in: International Conference on Learning Representations. URL: <https://openreview.net/forum?id=HyeSin4FPB>.
- Hu, N., Duan, H., Li, S., Hu, C., 2026. DFSC: Error-controlled differentiable mittag-leffler propagation for fractional scientific machine learning. doi:10.48550/arXiv.2607.29038, arXiv:2607.29038. arXiv preprint arXiv:2607.29038.
- Hu, Y., Anderson, L., Li, T.M., Sun, Q., Carr, N., Ragan-Kelley, J., Durand, F., 2020. DiffTaichi: Differentiable programming for physical simulation, in: International Conference on Learning Representations. URL: <https://openreview.net/forum?id=SVQXoincM3>.
- IEEE Standards Association, 2024. IEEE 1012-2024: Standard for System, Software, and Hardware Verification and Validation. Technical Report. IEEE. URL: <https://standards.ieee.org/ieee/1012/12536/>.
- International Organization for Standardization, 2016. ISO/IEC 25023:2016: Systems and Software Engineering—Systems and Software Quality Requirements and Evaluation (SQuaRE)—Measurement of System and Software Product Quality. Technical Report. ISO. URL: <https://www.iso.org/standard/35747.html>.
- International Organization for Standardization, 2021. ISO/IEC/IEEE 29119-2:2021: Software and Systems Engineering—Software Testing—Part 2: Test Processes. Technical Report. ISO. URL: <https://www.iso.org/standard/79428.html>.
- International Organization for Standardization, 2023. ISO/IEC 25010:2023: Systems and Software Engineering—Systems and Software Quality Requirements and Evaluation (SQuaRE)—Product Quality Model. Technical Report. ISO. URL: <https://www.iso.org/standard/78176.html>.
- Kovachki, N., Li, Z., Liu, B., Azizzadenesheli, K., Bhattacharya, K., Stuart, A., Anandkumar, A., 2023. Neural operator: Learning maps between

- function spaces with applications to pdes. *J. Mach. Learn. Res.* 24, 1–97. URL: <https://www.jmlr.org/papers/v24/21-1524.html>.
- Lischke, A., Pang, G., Gulian, M., Song, F., Glusa, C., Zheng, X., Mao, Z., Cai, W., Meerschaert, M.M., Ainsworth, M., Karniadakis, G.E., 2020. What is the fractional laplacian? a comparative review of ten definitions. *J. Comput. Phys.* 404, 109009. doi:10.1016/j.jcp.2019.109009.
- Lu, L., Meng, X., Mao, Z., Karniadakis, G.E., 2021. DeepXDE: A deep learning library for solving differential equations. *SIAM Rev.* 63, 208–228. doi:10.1137/19M1274067.
- Lubich, C., 1986. Discretized fractional calculus. *SIAM J. Math. Anal.* 17, 704–719. doi:10.1137/0517050.
- National Aeronautics and Space Administration, 2024. Standard for Models and Simulations. Technical Report NASA-STD-7009B. National Aeronautics and Space Administration. URL: <https://standards.nasa.gov/standard/nasa/nasa-std-7009>.
- Rackauckas, C., Ma, Y., Martensen, J., Warner, C., Zubov, K., Supekar, R., Skinner, D., Ramadhan, A., Edelman, A., 2020. Universal differential equations for scientific machine learning. arXiv preprint arXiv:2001.04385 doi:10.48550/arXiv.2001.04385.
- Rackauckas, C., Nie, Q., 2019. Confederated modular differential equation APIs for accelerated algorithm development and benchmarking. *Adv. Eng. Softw.* 132, 1–6. doi:10.1016/j.advengsoft.2019.03.009.
- Raissi, M., Perdikaris, P., Karniadakis, G.E., 2019. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.* 378, 686–707. doi:10.1016/j.jcp.2018.10.045.
- Tanyu, D.N., et al., 2023. Deep learning methods for partial differential equations and related parameter identification problems. *Inverse Probl.* 39, 103001. doi:10.1088/1361-6420/ace9d4.
- Virtanen, P., et al., 2020. SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nat. Methods* 17, 261–272. doi:10.1038/s41592-019-0686-2.

Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., Teal, T.K.,
2017. Good enough practices in scientific computing. PLOS Comput. Biol.
13, e1005510. doi:10.1371/journal.pcbi.1005510.